SOLID Design Principles

SOLID is an acronym for five design principles intended to make software designs more understandable, flexible, and maintainable over time. Adhering to these principles helps developers create systems that are easier to scale, test, and refactor.

S — Single Responsibility Principle (SRP)

A class should have only one reason to change. It must have a single, well-defined job.

O — Open/Closed Principle (OCP)

Software entities should be open for extension, but closed for modification. New functionality is added by writing new code, not by changing existing, working code.

L — Liskov Substitution Principle (LSP)

Objects of a superclass should be replaceable with objects of a subclass without breaking the application. Subclasses must adhere to the contract of their base class.

I — Interface Segregation Principle (ISP)

Clients should not be forced to depend on interfaces they do not use. It is better to have many small, specialized interfaces than one large, general-purpose one.

D — Dependency Inversion Principle (DIP)

High-level modules should not depend on low-level modules; both should depend on abstractions. Abstractions (interfaces) should dictate the contract, not concrete implementations (details).